



Lost and Found Management System

Team Member

- Md.Rayhan Kamal - 2024100000250
- Taraq Aziz - 2024100000244
- Arman Hossain Sami - 2024100000448
- Saiful Siddiky Labib - 2024100000469
- Alid Hossain Shuvo - 2024100000072

Section: 04

Course Code: CSE282

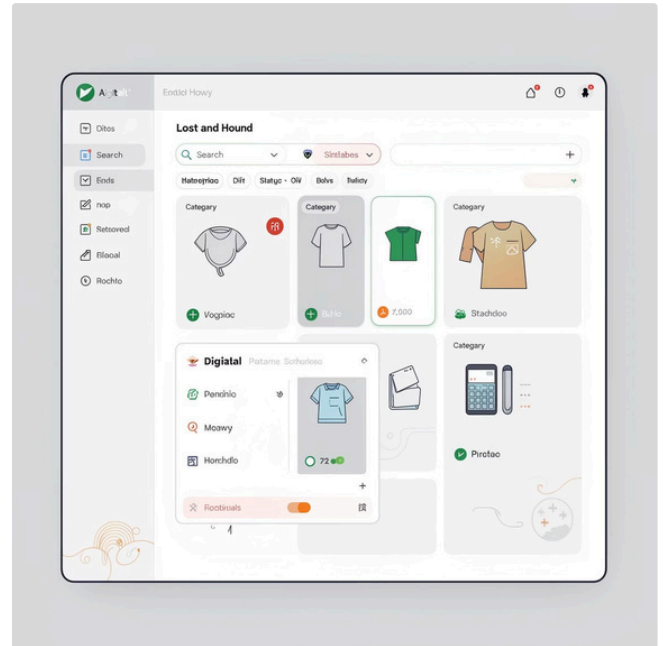
Prepared for: Md. Ayon Mia

Date: May 6, 2026

Project Introduction & Objective

TheLost& FoundManagementSystem (LFMS) is a desktop application built in Java, designed to solve a real and recurring problem at universities: the lack of a structured, centralized system for reporting and recovering lost items.

Currently, lost items at most campuses are handled informally — notices pinned to boards, word-of-mouth, or social media posts — with no reliable way to match a lost item report with a found item. This project addresses that gap directly.



Report Items

Students can report lost or found items via a clean GUI



Search Records

Search by keyword, category, or date



Submit Claims

Users can submit claim requests for items they believe belong to them



Admin Review

Admins review records, verify claims, and approve or reject them

The project is built entirely using Java, with Java Swing for the graphical interface, demonstrating all four OOP principles, structured exception handling, and organized data management.

Problem Statement

Institutions like universities deal with a high volume of lost and found items daily — wallets, phones, ID cards, bags, keys, and more. Without a centralized digital system, several critical issues arise:

Disconnected Reporting

Items are reported in disconnected places (notice boards, WhatsApp groups, verbal announcements)

Unreliable Matching

Matching lost reports to found items is slow, unreliable, and often fails entirely

No Accountability

There is no accountability — no record of who found what, or who claimed what

No Verification Tool

Admins have no tool to verify the legitimacy of claims



Solution: LFMS provides a single, organized platform where all lost and found records are stored, searchable, and managed through a defined claim-approval workflow.

OOP Concept Implementation

This project is designed to clearly demonstrate all four Object-Oriented Programming principles as required by the course:

Encapsulation

Private fields in Item, User, ClaimRequest with public getters/setters; internal state hidden from outside

Abstraction

Abstract class Item defines common structure (id, name, date, description) without exposing implementation

Inheritance

LostItem and FoundItem both extend Item, inheriting shared attributes while adding specific fields

Polymorphism

displayDetails() method overridden in LostItem and FoundItem to show type-specific information

Class Structure & Design

The system is organized around a clean class hierarchy. Every class has a defined responsibility, and all relationships follow OOP design principles.

Class	Key Variables / Fields	Key Methods / Functions
Item (Abstract)	id, name, description, date, category	displayDetails(), getId(), getName(), getDate(), setName()
LostItem extends Item	lostLocation, rewardOffered, contactInfo	displayDetails(), getLostLocation(), setRewardOffered()
FoundItem extends Item	foundLocation, currentHolder, storedAt	displayDetails(), getFoundLocation(), setStoredAt()
User	userId, name, email, phone, password	registerItem(), searchItems(), submitClaim(), login()
Admin extends User	adminId, role	viewAllRecords(), approveClaim(), rejectClaim(), deleteRecord()
ClaimRequest	claimId, userId, itemId, status, date	submitRequest(), updateStatus(), getStatus(), validate()

Abstract Base Class

Item is abstract, it cannot be instantiated directly. You must create either a LostItem or a FoundItem.

Inheritance Chain

Admin extends User, admins have all user capabilities plus privileged operations.

Association Link

ClaimRequest links a User to an Item, it stores the claim status and handles validation logic internally.

Key Methods & Functions

Item Management

ItemService class

- `addItem(Item item)` — validates input fields, checks for duplicates, stores the item
- `searchItems(String keyword, String category)` — filters records by name, category, or date range
- `updateItem(String id, Item updated)` — updates an existing record after validation
- `deleteItem(String id)` — removes a record; admin-only

Claim Management

ClaimService class

- `submitClaim(ClaimRequest req)` — validates and registers a new claim; throws `InvalidClaimException` if item is already claimed
- `approveClaim(String claimId)` — admin approves; marks item as returned
- `rejectClaim(String claimId)` — admin rejects with optional reason
- `getClaimsByUser(String userId)` — retrieves all claims made by a specific user

GUI Methods (Swing Panels)

- **`initComponents()`**
Initializes all UI elements (buttons, fields, tables)
- **`onSubmitClaim()`**
Reads form, calls `ClaimService`, shows success or error dialog
- **`onSearchClicked()`**
Reads search fields, calls `ItemService`, populates `JTable` with results
- **`refreshTable()`**
Reloads item list from the backend into the Swing table

Data, Variables & I/O

Core Variables

- Item: String id, name, description, category; LocalDate date
- LostItem: String lostLocation, contactInfo; boolean rewardOffered
- FoundItem: String foundLocation, storedAt; String currentHolder
- User: String userId, name, email, phone; String passwordHash
- ClaimRequest: String claimId, userId, itemId; ClaimStatus status; LocalDate submittedDate

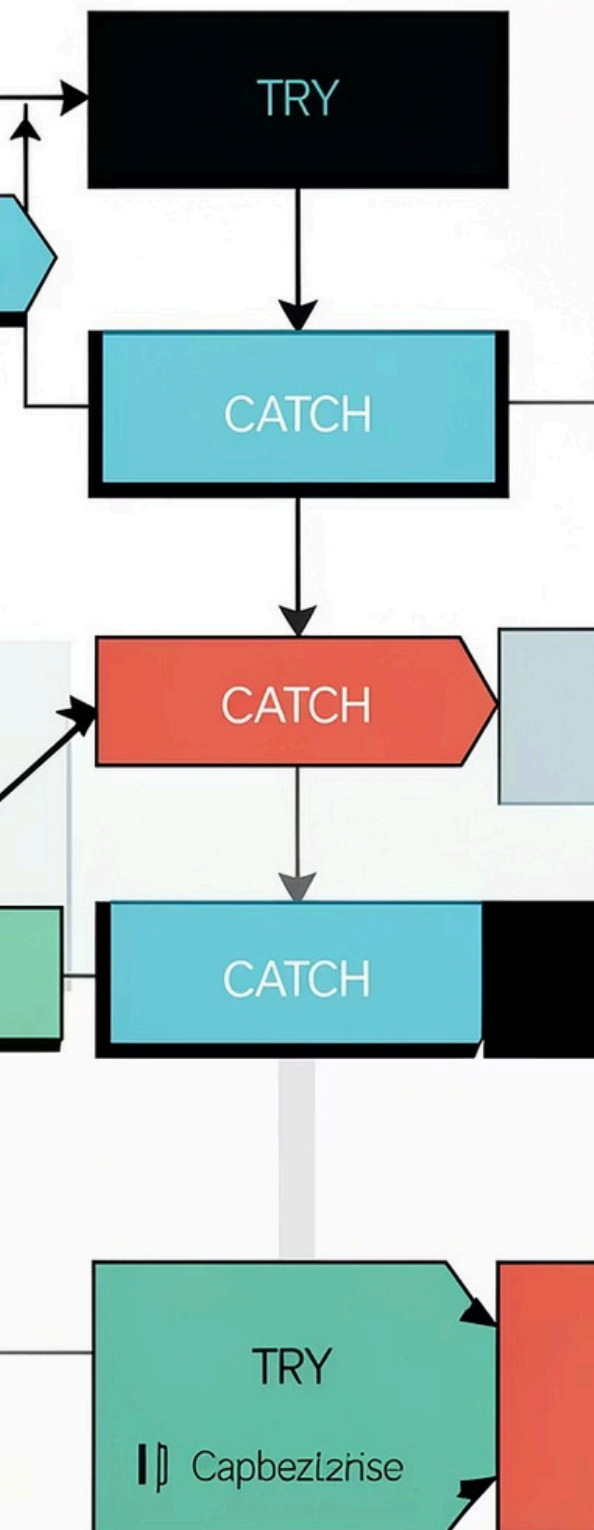
Inputs (from GUI forms)

- Lostitemform: item name, description, category, date lost, location, contact info, reward flag
- Found item form: item name, description, category, date found, location found, where stored now
- Search form: keyword (String), category (dropdown), date range (DatePicker)
- Claim form: user ID, item ID, reason for claim (text area)

Outputs

- Searchresults displayed in a scrollable JTable with columns: ID, Type, Name, Category, Date, Location
- Success/error dialogs via JOptionPane
- Admin dashboard showing all claims with status indicators (PENDING / APPROVED / REJECTED)
- Console/log output for debugging and audit trail

EXCEPTION HANDLING



Exception Handling Strategy

All user inputs are validated before processing. Errors are caught and displayed meaningfully to the user — no raw stack traces will be shown in the GUI.



DuplicateItemException

Thrown when a user attempts to register an item with matching name + date + location



InvalidClaimException

Thrown when a claim is submitted for an already-approved or non-existent item



UserNotFoundException

Thrown during login or claim submission if user ID is invalid



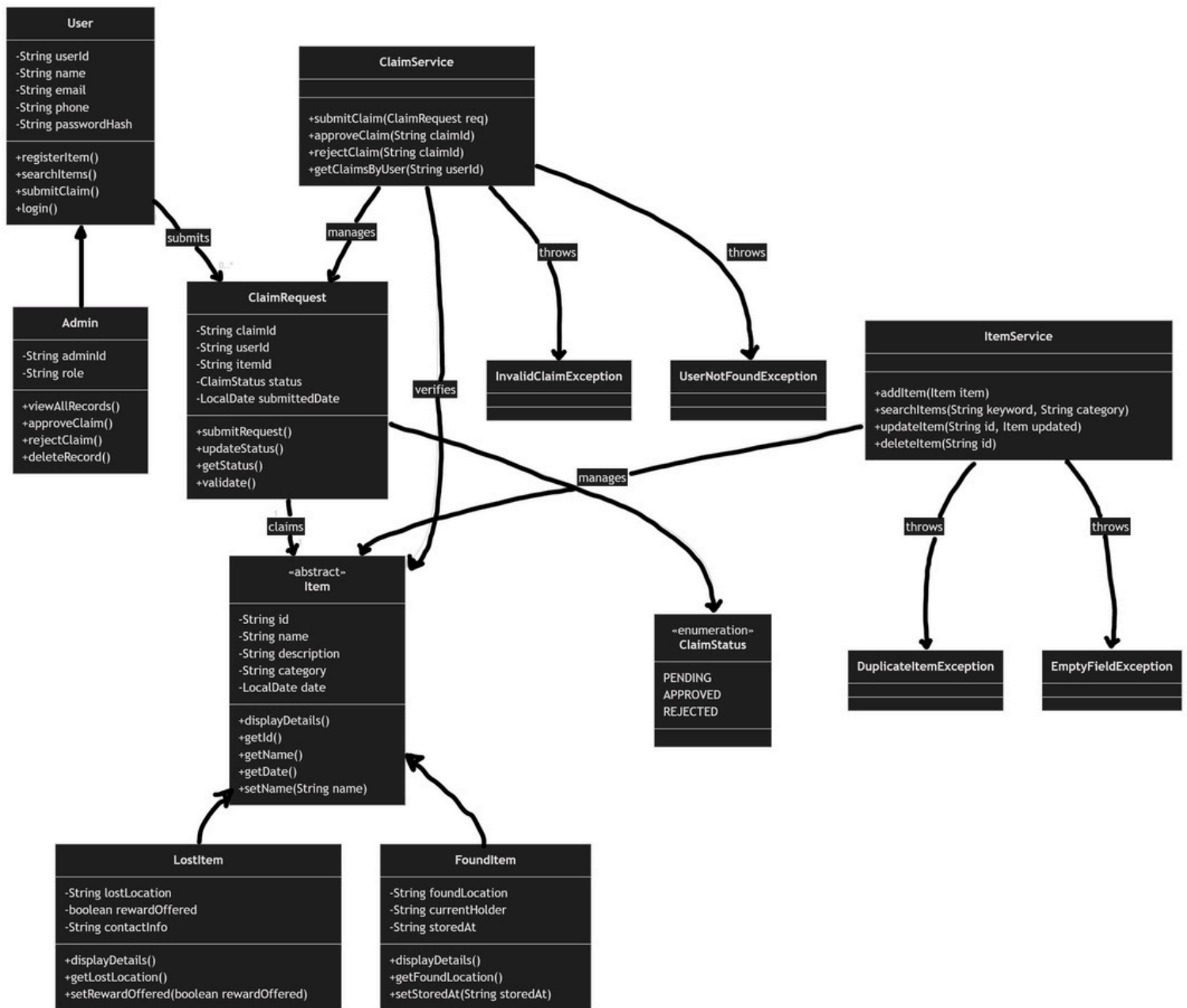
EmptyFieldException

Thrown when required form fields are left blank



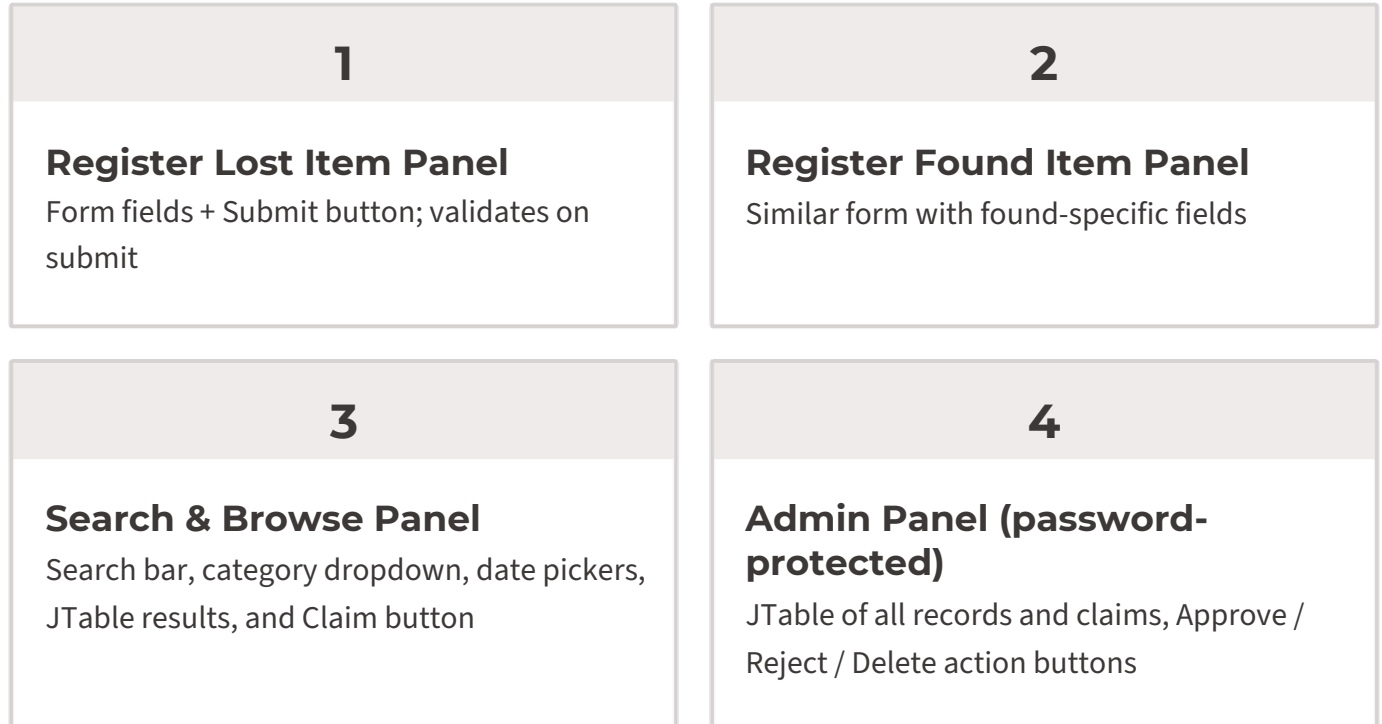
Implementation: try-catch blocks wrap all service calls in GUI event handlers;
`JOptionPane.showMessageDialog()` displays friendly error messages

Tentative Mermaid Diagram



GUI Layout (Java Swing)

The application uses a JTabbedPane as the main container with four panels:



All panels handle exceptions with try-catch and display user-friendly messages. The interface uses standard Swing components: JLabel, JTextField, JComboBox, JButton, JTable, JScrollPane, JOptionPane.

Conclusion

The Lost & Found Management System is a practical, well-scoped Java project that maps directly to the four course tasks. It demonstrates all required OOP principles through a real class hierarchy (Item → LostItem / FoundItem, User → Admin), implements structured exception handling through custom exceptions and try-catch blocks in the service layer, uses Java Swing to build a multi-panel GUI frontend, and the final report will document the complete development process.

The project is purposefully chosen because it solves a genuine, everyday problem at universities, making the work meaningful beyond just satisfying academic requirements.